Smart Cloud Cost optimization

USING MACHINE LEARNING

Introduction

1. Cloud computing is widely used today.

2.Cloud cost increase due to inefficient resource usage.

3.Manual monitoring is time-consuming.

4.Machine learning helps in smart cost optimization.

Proposed System

✅Uses “Machine Learning techniques”

1. Cloud Usage Prediction 📊

- The system analyzes historical cloud usage data (CPU, memory, traffic) using machine learning to forecast future resource requirements.

Example: The system predicts that next week’s workload will need 50% more CPU and memory.

2. Idle and Underutilized Resource Detection 🔍

- The system minimizes cloud cost through idle resource detection, rightsizing, and ML-based predictive scaling.

Example: Before optimization: ₹10,000/month
After optimization: ₹6,200/month
Savings achieved: 38%

3. Cloud Cost Optimization 💰

- The system automatically adjusts cloud resources based on predictions and usage analysis to reduce overall cost.

Example: Idle virtual machines are shut down or resized to lower-cost instances.

Proposed System

✅Uses “Machine Learning techniques”

4.🤖AI Chatbot for Cloud Insights

- An AI-based chatbot that allows users to ask questions about cloud usage and cost, and receive instant insights.

Example:

User asks: “Why is cost high this month?”

The system replies: “CPU usage increased by 40% during peak hours.

5. 📊 AI-Based Service Usage Ranking

- Analyzes cloud service expenses and ranks them based on their cost usage to identify the highest spending services

---

Example

🥇 Compute – ₹30,000

🥈 Storage – ₹15,000

🥉 Network – ₹5,000

Adavantages of Proposed System ✅

- Intelligent system using Machine Learning
- Accurate prediction of future cloud usage
- Automatic detection of idle resources
- Smart and dynamic resource optimization
- Significant reduction in cloud cost

System Architecture

Flow Diagram (Top → Bottom)

Phase 1

Cloud Usage Data 📊: Collect historical CPU, memory, traffic, and cost metrics.

Phase 2

Data Preprocessing ⚙️: Clean, normalize, and prepare data for ML models.

Phase 3

ML Prediction 🤖: Forecast future resource requirements using machine learning.

Phase 4

Cost Optimization 💰: Automatically resize or shut down resources to reduce cloud costs.

🔧 Hardware Requirements:

- System: Laptop / Desktop PC
- Processor: Intel i3 / i5 or higher (for smooth ML processing)
- RAM: 8 GB minimum, 16 GB recommended (for handling large cloud datasets)
- Storage: 100 GB or more (to store datasets, models, and database files)
- Internet Connection: Required to access cloud platforms and retrieve usage data

🖥️Software Requirements:

- Operating System: Windows 10 / 11
- Programming Language: Python (for data processing & ML implementation)
- IDE/Tools: VS Code / Jupyter Notebook (for coding and testing)
- Database: MySQL / SQLite (to store cloud usage data & reports)

⚙️Supporting Software & Platforms:

- Machine Learning Library: Scikit-learn (for prediction & analysis)
- Data Analysis Libraries: NumPy & Pandas (for numerical and data manipulation tasks)
- Cloud Platforms: Amazon Web Services / Microsoft Azure (to collect CPU, memory & storage metrics)
- Web Browser: For accessing the dashboard interface

Conclusion & Future Scope

✅ CONCLUSION

- ML-BASED SYSTEM IMPROVES CLOUD COST MANAGEMENT 📊.
- REDUCES RESOURCE WASTAGE ♻️.
- WORKS IN REAL-WORLD CLOUD ENVIRONMENTS ☁️.

🚀 FUTURE ENHANCEMENTS

- REAL-TIME CLOUD MONITORING ⏱️.
- MULTI-CLOUD OPTIMIZATION 🌐.
- REINFORCEMENT LEARNING FOR SMARTER ALLOCATION 🤖.

.

Digital Red Flag Identifier for Women

System Testing

System testing is a crucial phase in the development of the Digital Red Flag Identifier for Women. It ensures that the entire application functions correctly, securely, and efficiently according to the defined requirements. The purpose of system testing is to verify that all modules, including authentication, user dashboard, admin dashboard, category management, and red flag management, work together without errors. This phase validates both functional and non-functional aspects of the system before deployment.

Testing was conducted systematically to identify bugs, validate system behavior, and ensure that role-based access control is properly implemented. Each feature was tested individually and then tested collectively to confirm complete system integration.

1. Unit Testing

Unit testing was performed to verify the correctness of individual components. Each module such as login validation, category creation, red flag insertion, editing, and deletion was tested independently.

- Login module was tested with valid and invalid credentials.
- Category addition was tested with correct inputs and empty fields.
- Red flag insertion was tested to ensure proper linkage with selected categories.
- Update and delete operations were tested to confirm correct database modification.

This testing ensured that each function operates as expected without depending on other modules.

2. Integration Testing

After unit testing, integration testing was performed to verify communication between different modules.

- Frontend forms were tested to ensure data is correctly sent to backend controllers.
- Backend services were tested to confirm proper interaction with the MySQL database.
- Category selection was tested to verify that only related red flags are displayed.

- Role-based redirection was verified after successful login.

Integration testing ensured that data flows smoothly between the presentation layer, application layer, and database layer.

3. Functional Testing

Functional testing was conducted to ensure that the system performs all intended operations correctly.

Authentication Testing:

- Valid email and password → Successful login.
- Invalid credentials → Proper error message displayed.
- Role-based redirection → User dashboard for users, Admin dashboard for administrators.

User Module Testing:

- Viewing category list.
- Selecting a category.
- Viewing red flags under selected category.
- Logout functionality.
- Ensuring no edit or delete options are available to users.

Admin Module Testing:

- Add new category.
- Edit category name.
- Delete category.
- Add new red flag with required details.
- Edit existing red flag.
- Delete red flag.
- Logout functionality.

All workflows were tested to confirm that they align with the defined system requirements.

4. Security Testing

Security testing was performed to ensure the protection of system data and controlled access.

- Unauthorized URL access was tested to confirm users cannot directly access admin pages.
- Session validation was checked to prevent page access after logout.
- Input validation was implemented to avoid empty or invalid submissions.
- Database integrity was verified to prevent inconsistent or orphan records.

This testing ensures that only authorized users can perform administrative operations and that sensitive data is protected.

5. Usability Testing

Usability testing was conducted to verify that the system is user-friendly and easy to navigate.

- Navigation between pages was tested for clarity.
- Buttons and links were checked for proper functioning.
- Content readability was verified.
- Error messages were reviewed for clarity and helpfulness.

The goal was to ensure that female users can easily understand and access awareness content without technical difficulty.

6. Performance Testing

Basic performance testing was conducted to ensure the system responds efficiently.

- Login response time was tested.
- Category and red flag retrieval speed was checked.
- Multiple CRUD operations were tested to confirm consistent performance.

The system was verified to handle normal usage without delay or data inconsistency.

7. Test Case Validation

Test cases were prepared for each module covering both positive and negative scenarios. Expected outputs were compared with actual outputs to confirm system accuracy. All identified issues were corrected and retested to ensure reliability.\

Conclusion of System Testing

System testing confirmed that the Digital Red Flag Identifier for Women functions according to its requirements. All modules including authentication, category management, red flag management, and role-based access control were tested successfully. The system demonstrated reliability, security, usability, and proper integration between components. After successful completion of testing, the system was ready for deployment and practical use.